Sc: 6
Date: 6/3/26

Error Detection & correction - Hamming code.

Aim:- To write a program to implement error detection & correction using Hamming code.

Algorithm:-

1) start the program
2) Read the number of 4 bits data blocks (n) forming the input data stream.
3) For each 4 bit data block $D = d_1 \& d_2, d_3, d_4$ perform hamming coding.
   a) Arrange 7 bits with parity bits at position 1, 2 and 4.
   b) compute the parity bits using even parity.

$P_1 = d_1 \text{ XOR } d_2 \text{ XOR } d_4$ (cover bit - 1, 3, 5, 7)

$P_2 = d_1 \text{ XOR } d_3 \text{ XOR } d_4$ (cover bit - 2, 5, 6, 7)

   c) form the 7 bit hamming code word: P_1 P_2 d_1 P_4 d_2 d_3 d_4
4) Simulate transmission by introducing a single bit error at a chosen bit position in the code word.

5) At receiver, recompute the check bits (syn drome) from received code word.

$c_1 = P_1 \text{ XOR } d_1 \text{ XOR } d_2 \text{ XOR } d_4$

$c_2 = P_2 \text{ XOR } d_1 \text{ XOR } d_3 \text{ XOR } d_4$

$c_4 = P_4 \text{ XOR } d_2 \text{ XOR } d_3 \text{ XOR } d_4$

6) compute error position = $c_4 c_2 c_1$ (binary no formed)

7) If error position = $c_4 c_2 c_1$ (binary no formed)

8) Extract the original 4 data bits from corrected 7 bit codes.

9) Repeat step 3 to 8 for all n blocks in the data storage.

10) stop the program

Program:

```
def encode(d):
    """
    d : list of 4 bits [d1, d2, d3, d4] -> returns 7 bit
    Hamming code list position (1-indexed)
    1 = P1, 2 = P2, 3 = d1, 4 = P4, 5 = d2,
    6 = d3, 7 = d4
    """
    h = [0] * 7
    h[2] = d[0]  # position 3
    h[4] = d[1]  # position 5
    h[5] = d[2]  # position 6
    h[6] = d[3]  # position 7
```

```
h[0] = h[2] ^ h[4] ^ h[6]  # P1 covers 1,3,5,7
h[1] = h[2] ^ h[5] ^ h[6]  # P2 covers 2,3,6,7
h[3] = h[4] ^ h[5] ^ h[6]  # P4 covers 4,5,6,7

return h
def delete and correct (h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
error_pos = c1*1 + c2*2 + c4*4  # Syndrome -> faulty bit position

if error_pos != 0
    print("Error detected at bit position:
          " error_pos)
    h[error_pos - 1] ^= 1

else:
    print("No error detected")

    return error_pos
def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    n = int(input())
    for b in range(1, n+1):
        print("block {b}")
        d = list(map(int, input().split()))
        h = encode(d)
```

output :-

=== Hamming code - error Detection and correction ===

Enter number of 4 bit data blocks in the stream: 1

--- Block 1 ---

Enter 4 data bits (space Separated .eg .1011): 0 1 0 1

original Data bits : 0 1 0 1

Encoded Hamming code (7 bits) : 0 1 0 0 1 0 1

Enter bit position to introduce error (1-7, 0=no error): 3

Transmitted code (with error induced at pos 3): 0 1 1 0 1 0 1

Recieved code : 0 1 1 0 1 0 1

checking code : 0 1 0 0 1 0 1

Recovered Data Bits : 0 1 0 1

--- Block 2 ---

Enter 4 data bits (space separated, Eg .1011): 1 1 0 1

original Data Bits : 1 1 0 1

Encoded Hamming code (7 bits) : 1 0 1 0 1 0 1

Enter bit position to introduce error (1-7, 0= no error): 5

Transmitted code (with error induced at pos 5) :

1 0 1 0 0 0 1

```
print("original data bits: {to_str(d)}")
print("Encoded Hamming code (7 bits):
                {to_str bin}")
pos = int(input("Enter bit to introduce
        error (1-7), 0 = "no error"))

if 1 ≤ pos ≤ 7:
    h[pos-1] ^= 1
    print("Transmitted code: {to_str(h)}")

else:
    print("Transmitted code: {to_str(h)}")
print("Recieved code"; {to_str(h)})
print("checking error...")
    detect error and correct(h)
    print("corrected code: {to_str(h)}")
rd = [h[2], h[4], h[5], h[6]]
print("recovered data bits: {to_str
                (rd)}")
if __name__ == "__main__":
    main()
```

Result :- The program to implement error detection & correction using hamming code heart was successfully written in python, executed and verified with a sample input data stream.

Received code : 1 0 1 0 0 0 1

checking for error...

Error detected at bit position : 5 -> correcting...

corrected code : 1 0 1 0 1 0 1

Recovered code : 1 1 0 1